TuHospedaje

INFORME DE SPRINT

# Bitácora de Ejecución y Cierre — Sprint 4

TuHospedaje — Reservas, Historial, WhatsApp, Email, Panel de Administración, Autenticación Segura y Cancelación de Reservas

HISTORIAS
US #30 a #35

INTEGRADO EN MAIN
2026-07-23 (PR #36)

E2E
45 escenarios en 13 specs

Equipo de Desarrollo · Junio-Julio 2026

**Foco del Incremento:** Reservas, Historial y Comunicación (Incremento 1) · Tablas Uniformes y Dashboard de Reservas (Incremento 2) · Refactor de Arquitectura y Mejoras (Incremento 3) · Autenticación Segura, Cancelación de Reservas y Confiabilidad de Frontend (Incremento 4) **Stack Tecnológico:** Java 17 / Spring Boot 3.5 / Spring Security 6 / JavaMailSender / MariaDB / React 19 / Vite / Testcontainers / SpringDoc OpenAPI

Este documento consolida los cuatro incrementos ejecutados sobre la rama `sprint-4` : el alcance original del sprint (reservas, WhatsApp, email), la consolidación posterior del panel de administración (previamente documentada como "Sprint 4.5"), el refactor de arquitectura, y el endurecimiento de autenticación junto con la cancelación de reservas y mejoras de confiabilidad del frontend. Se presentan de forma unificada, no como reportes separados, para reflejar el estado final del incremento. El corte histórico quedó integrado a `main` mediante el merge commit `8a3fd43` (PR #36) el 23 de julio de 2026; el estado vigente de `main` incluye los PR #73 y #75 y se referencia en la Sección 9.

## 1. Resumen del Incremento (Scope)

### 1.1. Incremento 1 — Reservas, Historial y Comunicación

El Sprint 4 completó la funcionalidad central de reservas de la plataforma. Se reemplazó el modal de reserva provisional (Sprint 3) por un flujo de página dedicada en dos columnas ( `/booking/:lodgingId` ), que presenta el resumen completo del alojamiento junto al formulario de reserva. Al confirmar, el usuario es redirigido a una pantalla de confirmación con los detalles de la estadía.

Se implementó el historial personal de reservas ( `/my-reservations` ) con ordenamiento por fecha de entrada descendente, protegido por el nuevo guard `RequireAuth` . Se incorporó un botón flotante de WhatsApp configurable por variable de entorno, y se desarrolló un servicio de email SMTP real ( `SmtpEmailServiceImpl` ) que envía confirmaciones HTML al huésped mediante Mailtrap, activable de forma independiente al `ConsoleEmailServiceImpl` ya existente.

En el backend, se corrigió el endpoint de disponibilidad para retornar los rangos ocupados ( `occupiedRanges` ) sin necesidad de recibir fechas, permitiendo el bloqueo visual del calendario al cargar la página. Se agregó validación de reserva confirmada antes de permitir puntuaciones ( `RatingServiceImpl` ), y se añadieron 5 nuevos tests de integración con Testcontainers.

Como mejora complementaria al panel de administración, se extendió `LodgingFormModal` con soporte de edición (prop `lodging` opcional + `PUT` ) y se incorporaron los botones "Editar" por fila en la tabla de alojamientos.

Adicionalmente, como agregado por iniciativa del equipo fuera del alcance original del sprint, se incorporó una suite de pruebas end-to-end con Playwright (carpeta `e2e/` ), que valida los flujos críticos de la aplicación en Chromium y Firefox.

### 1.2. Incremento 2 — Tablas Uniformes y Dashboard de Reservas

Este incremento consolidó la experiencia de usuario en el Panel de Administración mediante dos ejes principales: la unificación de tablas y la visualización de reservas recientes.

Se estandarizó la lógica de presentación de datos mediante el hook personalizado `useTableData` y los componentes reutilizables `SortableTh` (encabezados ordenables) y `Pagination` (control de páginas), integrados en las vistas de Categorías, Características, Políticas y Usuarios. Para mantener la consistencia del panel, se migró la tabla de alojamientos ( `AdminLodgings` ) de paginación del servidor a paginación del cliente ( `GET /api/lodgings` plano), agregando además la columna de descripción. El componente de paginación unificado también se integró en `SearchResults` en este momento (antes de que el Incremento 3 lo hiciera server-driven nuevamente para ese caso puntual — ver 1.3).

Se implementó el soporte para visualizar las reservas recientes en el Dashboard del administrador: el endpoint `GET /api/reservations` (exclusivo ADMIN) retorna todas las reservas ordenadas de forma descendente por ID, y la UI del Dashboard se enriqueció con una tarjeta de estadísticas de "Reservas" y una sección "Últimas reservas" con las primeras 4 transacciones.

### 1.3. Incremento 3 — Refactor de Arquitectura y Mejoras

Cierre de deuda técnica identificada durante el desarrollo del panel de administración, ejecutado mediante un ciclo SDD completo (propuesta → especificación → diseño → tareas → implementación) y entregado en 6 pull requests encadenadas.

**Backend:** se reemplazaron los campos `Map<String, Object>` de `features` / `policies` en `LodgingDTO` por DTOs tipados ( `FeatureSummaryDTO` , `PolicySummaryDTO` ). El endpoint `GET /api/lodgings/search` pasó de devolver un array plano a soportar paginación real ( `page` / `size` ) y filtrado por múltiples categorías directamente en base de datos (expresión `IN` sobre la `Specification` JPA existente, reemplazando el filtrado en memoria que hacía el frontend) — la respuesta ahora usa el mismo formato paginado que ya utilizaba `GET /api/lodgings` . Se agregó internacionalización ( `Accept-Language` ) acotada a 4 de los 9 exception handlers de `GlobalExceptionHandler` ( `ResourceNotFoundException` , `IllegalArgumentException` , y los nuevos handlers de validación de parámetros `ConstraintViolationException` / `HandlerMethodValidationException` ), con bundles `messages.properties` / `messages_es.properties` .

**Frontend:** se creó una capa de servicios por dominio ( `lodgingService.js` , `categoryService.js` , `favoriteService.js` ) para desacoplar las llamadas HTTP directas de los componentes. `SearchResults.jsx` se reescribió para consumir esta capa, eliminando la paginación client-side y el filtrado en memoria de múltiples categorías en favor del nuevo contrato paginado del servidor. `RequireAdmin` se convirtió en un layout basado en `<Outlet/>` (alineado con el patrón ya usado por `RequireAuth` ), y el redirect para usuarios autenticados sin rol ADMIN cambió de `/` a una nueva página `/unauthorized` .

**Nota histórica sobre paginación:** el Incremento 2 documentó como deuda técnica controlada "migrar [alojamientos] a paginación por base de datos únicamente si la volumetría de producción lo requiere". El Incremento 3 resolvió esto específicamente para `/api/lodgings/search` (el endpoint público, donde el filtrado en memoria de múltiples categorías ya era una ineficiencia medible), **no** para las tablas administrativas. El resto de esa afirmación describe la decisión histórica del Incremento 2; el estado actual de `AdminLodgings` está documentado en la Sección 9.

### 1.4. Incremento 4 — Autenticación Segura, Cancelación de Reservas y Confiabilidad de Frontend

Este incremento reemplazó el esquema de autenticación basado en JWT en el cuerpo de la respuesta (almacenado en `localStorage` por el cliente) por uno de cookie `HttpOnly` con protección CSRF, agregó la posibilidad de que un cliente cancele su propia reserva antes del check-in, y mejoró la resiliencia de la carga de rutas del frontend.

#### 1.4.1. AUTENTICACIÓN BASADA EN COOKIE `HTTPONLY` Y PROTECCIÓN CSRF

El JWT ya no viaja en el cuerpo de la respuesta ni se almacena en `localStorage`: `AuthController` lo entrega en una cookie `ACCESS_TOKEN` `HttpOnly` (`AuthCookieFactory`, nueva), inaccesible desde JavaScript — mitiga la exfiltración del token ante un XSS. Esto exige protección CSRF para las mutaciones, implementada con el patrón *double-submit cookie* recomendado por Spring Security para SPAs: `SpaCsrfTokenRequestHandler` (nuevo) combina resolución XOR y directa según si la request trae el header `X-XSRF-TOKEN`, y materializa el token de forma eager en cada respuesta salvo en el bootstrap anónimo de `GET /api/auth/csrf`.

`JwtAuthenticationFilter` pasó a leer el JWT desde la cookie `ACCESS_TOKEN` en vez de un header `Authorization`, y degrada a no autenticado (sin propagar la excepción) ante un JWT inválido o un usuario borrado después de emitido — corrigiendo un 500 espurio a 401/403. Se agregaron los endpoints `GET /api/auth/me` (identidad de la sesión), `POST /api/auth/logout` (limpia la cookie, requiere CSRF válido) y `GET /api/auth/csrf` (bootstrap explícito del token). El frontend (`api.js`, `AuthContext.jsx`) migró a `credentials: "include"` + header `X-XSRF-TOKEN`, y secuencia login/registro para esperar el bootstrap de CSRF antes de publicar el estado autenticado — evita que la UI muestre "sesión iniciada" antes de que exista una cookie CSRF utilizable.

Durante la verificación de este incremento se detectó y corrigió un defecto real: bajo `SessionCreationPolicy.STATELESS` (sin `HttpSession`), la estrategia por defecto de Spring Security (`CsrfAuthenticationStrategy`) rota la cookie CSRF en **cada** request autenticado, no solo en el login, porque no hay sesión donde recordar "ya procesado". Esa rotación competía con los requests paralelos que dispara la SPA después del login, causando cierres de sesión intermitentes con CSRF inválido. Se reemplazó por `NullAuthenticatedSessionStrategy` (correcto en una app sin sesión) y se agregó una rotación puntual y atómica del token exactamente en login/registro, para preservar la única propiedad de seguridad real que la estrategia por defecto aportaba (invalidar un token plantado antes del login).

#### 1.4.2. BASE DE SESIONES RENOVABLES (ESTADO HISTÓRICO DEL REPORTE ORIGINAL)

En el estado documentado originalmente, la infraestructura de sesiones renovables se implementó con rotación y detección de replay: entidades `RefreshToken`, `RefreshTokenFamily` y `SessionSecurityEvent`; `RefreshSessionService` / `Impl` para emisión, rotación, revocación, bloqueo de familias y límites de tasa; `RefreshTokenHasher`; y la migración `V2__refresh_session_families.sql`. En ese momento, la base estaba probada de forma aislada y **no estaba conectada a un endpoint real**; `app.session.refresh.enabled=false` y los JWT de acceso estaban gobernados exclusivamente por `app.jwt.expiration`. El estado posterior se registra en el addendum de la Sección 9.

#### 1.4.3. CANCELACIÓN SELF-SERVICE DE RESERVAS

Se agregó `PATCH /api/reservations/{id}/cancel` (`@PreAuthorize("isAuthenticated()")`), que permite a un usuario cancelar su propia reserva mientras esté `CONFIRMED` y la fecha de check-in no haya llegado, evaluado con un `Clock` de negocio fijado a `America/Argentina/Buenos_Aires` (`TimeConfiguration.businessClock`) en vez del reloj del sistema — evita que el corte dependa de la zona horaria del servidor. El servicio usa un lock pesimista de escritura (`findByIdForUpdate`) para que cancelaciones concurrentes produzcan una única transición de estado y un único email; una reserva ya cancelada responde de forma idempotente (200, sin reenviar el email); una reserva ajena o inexistente responde con la misma excepción que `getReservationById` (mismo patrón anti-IDOR ya usado en el resto del proyecto); intentar cancelar en o después de la fecha de check-in devuelve 400 sin modificar el estado. El email de cancelación se envía tras el commit de la transacción (`TransactionSynchronization.afterCommit()`); un fallo de envío se loguea pero no deshace la cancelación. En el frontend, `MyReservationsPage.jsx` ofrece el botón solo para reservas `CONFIRMED` con check-in futuro, pide confirmación nativa (`window.confirm`), bloquea envíos duplicados por fila mientras la solicitud está en curso, y reemplaza únicamente la fila afectada en el estado local al finalizar.

Como parte de este trabajo se corrigió además un test backend intermitente en CI (`f606201`): construía su fixture con `LocalDate.now()` del sistema en vez del reloj de negocio, por lo que cerca de la medianoche en Buenos Aires (UTC-3) ambos relojes podían discrepar de fecha calendario y volver falso el supuesto "el check-in es hoy" del test. Se corrigió inyectando el mismo `Clock` de negocio que usa la aplicación.

#### 1.4.4. CARGA DIFERIDA DE RUTAS Y METADATA EN ESPAÑOL

Las 11 páginas de rutas de la aplicación (`Home`, `Login`, `Register`, `ProductDetail`, `Admin`, `SearchResults`, `Favorites`, `Booking`, `BookingConfirmation`, `MyReservations`, `Unauthorized`) pasaron a cargarse con `React.lazy()`, reduciendo el bundle inicial. La resiliencia agregada no es la sola división en chunks, sino dos componentes nuevos: `RouteChunkErrorBoundary` (límite de error de clase, con `getDerivedStateFromError`) captura fallos de carga de chunk — típicos tras un despliegue con chunks obsoletos en caché del navegador — y ofrece una recarga manual completa en vez de fallar en blanco, reseteándose automáticamente al cambiar de ruta; `RouteLoadingFallback` muestra un spinner accesible (`role="status"`) con un retraso de 150 ms antes de aparecer, para no parpadear en navegaciones rápidas, y respeta `prefers-reduced-motion`. La "metadata en español" es más acotada de lo que sugiere el nombre: `index.html` declara `lang="es"` y un `<title>` fijo; se descartó

deliberadamente introducir títulos dinámicos por ruta o una dependencia como `react-helmet` para este alcance, documentado explícitamente en el test correspondiente.

### 1.5. Actualización posterior al reporte original

- `AdminLodgings` volvió a paginación, ordenamiento y búsqueda dirigidos por el servidor. Las tablas administrativas de Categorías, Características, Políticas y Usuarios conservan el esquema client-side.
- `AdminReservations` incorporó consulta, filtrado, ordenamiento y paginación desde el servidor. La gestión de reservas continúa sin acciones administrativas de cancelación o reprogramación.
- Se localizaron las respuestas restantes de `GlobalExceptionHandler`, se corrigió la configuración de ESLint para tests y se eliminó la deuda de lint registrada en el reporte original.
- **Nota histórica sobre CI:** el workflow se configuró para ejecutarse en pushes y pull requests de `main` y `sprint-4`. Esa actualización describía el alcance del workflow y no afirmaba el resultado de una ejecución remota específica; la evidencia actual de CI está registrada en la Sección 9.
- Flyway pasó a administrar el ciclo de vida del esquema mediante una migración base. Hibernate valida el esquema y los datos de demostración se separaron en un seed de desarrollo versionado, descartable y de activación explícita.
- Se aprobaron 38 identidades visuales canónicas, una por alojamiento, en `content/lodgings/`. Cada identidad define cinco escenas, por lo que queda una producción pendiente de 190 imágenes.

### 1.6. Slice responsive móvil

El PR #73 incorporó el shell responsive compartido: navegación móvil con menú accesible, prevención de overflow y wrapping en anchos reducidos, y objetivos táctiles adecuados. El PR #75 extendió esa base a `MyReservations`, con tarjetas y acciones que se adaptan sin desbordarse y con la interacción de cancelación disponible desde móvil. La cobertura `mobile-chromium` verifica ambos slices en viewports de `390x844` y `320x844`, incluyendo menú accesible, wrapping, touch targets y confirmación/actualización de cancelación.

#### LÍMITE DE INTEGRACIÓN DE IMÁGENES

Los archivos JSON de `content/lodgings/` son la fuente canónica para construir prompts y verificar continuidad visual. La base de datos conserva únicamente la URL pública y el título de cada imagen mediante `lodging_images`; no almacena identidades, prompts ni archivos binarios. Las 190 URLs genéricas del seed de desarrollo son datos provisionales y deberán reemplazarse por URLs estables después de generar, revisar y publicar las imágenes canónicas.

## 2. Arquitectura del Sistema e Integración

### 2.1. Backend (Spring Boot + Spring Security 6)

Se expandió la arquitectura existente sin introducir nuevos módulos estructurales:

```
Controller → Service (Interface + Impl) → Repository → Entity / DTO
```

#### MATRIZ DE COMPONENTES INTRODUCIDOS O MODIFICADOS:

| MÓDULO | ENTIDAD / DTO AFECTADO | CAMBIO EN CAPA DE SERVICIO | CAMBIO EN CONTROLLER |
|---|---|---|---|
| **Reservas (Inc. 1)** | `Reservation` (+ `guestPhone`), `ReservationResponse`, `CreateReservationRequest` | `ReservationService` / `ReservationServiceImpl`: historial propio | `ReservationController`: `GET /api/reservations/my`, corrección de `availability` |
| **Email (Inc. 1)** | — | `SmtpEmailServiceImpl` (nueva, `@Primary`), `ConsoleEmailServiceImpl` (renombrado) | — |
| **Alojamientos (Inc. 1)** | `LodgingDTO` (+ `averageRating`, + `ratingCount`) | `LodgingServiceImpl.enrichWithRatings()` | — |
| **Categorías (Inc. 1)** | `Category` (+ `imageUrl`) | — | — |
| **Reseñas (Inc. 1)** | — | `RatingServiceImpl`: validación de reserva CONFIRMED previa | — |
| **Reservas — Admin (Inc. 2)** | `Reservation`, `ReservationResponse` | `ReservationServiceImpl`: todas las reservas ordenadas por ID desc | `ReservationController`: `GET /api/reservations`, `@PreAuthorize("hasRole('ADMIN')")` |
| **Alojamientos — DTOs tipados (Inc. 3)** | `LodgingDTO` (features/policies tipados), `FeatureSummaryDTO`, `PolicySummaryDTO` (nuevos) | `LodgingServiceImpl`: mapeo manual actualizado | — |
| **Alojamientos — Búsqueda (Inc. 3)** | — | `LodgingServiceImpl.search()`: `Specification` con `IN` sobre categorías, `Pageable`, retorna `Map` paginado | `LodgingController`: `/search` acepta `categories`, `page`, `size` validados (`@Validated` + `@Min`) |

| | | | |
|---|---|---|---|
| **Excepciones — i18n (Inc. 3)** | `ResourceNotFoundException` (+ `errorCode`, + `args`, retrocompatible) | — | `GlobalExceptionHandler`: 4 de 9 handlers localizados vía `MessageSource` / `Locale` |
| **Autenticación — cookie + CSRF (Inc. 4)** | — (sin cambios de entidad) | `AuthCookieFactory` (nueva), `JwtAuthenticationFilter` lee la cookie `ACCESS_TOKEN`, `SpaCsrfTokenRequestHandler` (nuevo), `CsrfTokenRepository` como bean compartido | `AuthController`: nuevos `GET /me`, `POST /logout`, `GET /csrf`; `SecurityConfig`: CSRF habilitado, `NullAuthenticatedSessionStrategy` |
| **Sesiones renovables — base (Incremento 4, estado histórico)** | `RefreshToken`, `RefreshTokenFamily`, `SessionSecurityEvent` (nuevas, migración `V2`) | `RefreshSessionService` / `Impl`, `RefreshTokenHasher` | Sin endpoint conectado en el corte original; `app.session.refresh.enabled=false` |
| **Reservas — cancelación (Inc. 4)** | `Reservation` (`CANCELLED` ahora alcanzable) | `ReservationServiceImpl.cancelReservation()`, lock pesimista (`findByIdForUpdate`), `TimeConfiguration.businessClock` (nuevo) | `ReservationController`: `PATCH /{id}/cancel` |

- **SmtpEmailServiceImpl con `@Primary`:** se optó por `@Primary` en lugar de `@ConditionalOnMissingBean` porque esta última anotación solo evalúa confiablemente en clases `@Configuration`. `@Primary` resuelve la ambigüedad de forma explícita y determinista.
- **Endpoint de disponibilidad corregido:** `GET /api/lodgings/{id}/availability` acepta `checkIn` / `checkOut` opcionales. Sin parámetros, devuelve todos los rangos CONFIRMED (`occupiedRanges`) para bloqueo visual del calendario.
- **Validación de reseñas:** `RatingServiceImpl.createRating()` verifica `existsByUserIdAndLodgingIdAndStatus(CONFIRMED)` antes de persistir.
- **Restricción de acceso a `/api/reservations` (Inc. 2):** `@PreAuthorize("hasRole('ADMIN')")`. Confirmado por integración: sin credenciales → 401, no-admin → 403, admin → 200 con listado ordenado por ID desc.
- **Búsqueda multi-categoría en base de datos (Inc. 3):** la `Specification` JPA existente (que ya filtraba por una única categoría con `equal`) se extendió a `root.get("category").get("id").in(categories)`, evitando múltiples queries o filtrado en memoria.
- **Validación de `page` / `size` con Bean Validation, no con excepciones ad-hoc (Inc. 3):** se evaluó usar el mensaje de la excepción como clave de traducción (`IllegalArgumentException`), pero se descartó porque ningún punto real del código (13 sitios existentes) usa ese patrón — todos lanzan texto plano en español. Se optó por `@Validated` + `@Min(message="{clave}")` en `LodgingController`, el mecanismo idiomático de Spring para localizar validaciones de parámetros. Es el primer controller del proyecto en usar `@Validated` a nivel de clase (el resto de las validaciones existentes son `@Valid` sobre `@RequestBody`, que no lo requiere).
- **Corrección de un bug de entorno real (Inc. 3):** `spring.messages.fallback-to-system-locale` (default `true` en Spring Boot) hacía que, en un host con locale del sistema operativo en español, pedir `Accept-Language: en` devolviera igualmente el mensaje en español — comportamiento no determinístico según el entorno de ejecución. Se deshabilitó explícitamente en `application.properties` (main y test).
- **Alcance acotado de la i18n (Inc. 3):** de los 9 exception handlers de `GlobalExceptionHandler`, solo 4 fueron localizados (`ResourceNotFoundException`, `IllegalArgumentException`, `ConstraintViolationException`, `HandlerMethodValidationException`). Los otros 5 (`AuthenticationException`, `MethodArgumentNotValidException`, `ObjectOptimisticLockingFailureException`, `PessimisticLockingFailureException`, `UploadException`, `DataIntegrityViolationException` y el catch-all genérico) mantienen sus mensajes en español hardcodeado — decisión explícita para no inflar el alcance del cambio (ver Sección 8).
- **Cookie `HttpOnly` sobre `localStorage` para el JWT (Inc. 4):** el token queda inaccesible desde JavaScript, mitigando su exfiltración ante un XSS; a cambio, exige protección CSRF explícita para las mutaciones (documentado como "Design Decision 1" en `AuthCookieFactory`).
- **Double-submit cookie con XOR + resolución directa (Inc. 4):** `SpaCsrfTokenRequestHandler` sigue el patrón oficial recomendado por Spring Security para SPAs, en vez de la variante de sesión clásica.
- **`NullAuthenticatedSessionStrategy` en vez del default `CsrfAuthenticationStrategy` (Inc. 4):** el default rota la cookie CSRF en cada request autenticado bajo `STATELESS`, no solo en el login, porque no hay `HttpSession` para recordar "ya procesado" — causaba cierres de sesión intermitentes. La estrategia de sesión correcta reemplazó el override en `SecurityConfig`, no en `SessionManagementConfigurer` (este último solo agrega a la misma lista compuesta que ya contiene `CsrfAuthenticationStrategy`, no la reemplaza).
- **Rotación de CSRF acotada a login/registro, atómica (Inc. 4):** para no perder la invalidación de un token plantado antes del login, se rota el token una sola vez ahí (`generateToken()` + `saveToken()` en una sola escritura). Una primera versión que limpiaba y regeneraba en dos pasos separados escribía dos encabezados `Set-Cookie` para el mismo nombre en una respuesta; el consumidor leía el primero (vacío), rompiendo el siguiente logout — corregido con una única escritura.
- **Lock pesimista para cancelación concurrente (Inc. 4):** `findByIdForUpdate` asegura que cancelaciones concurrentes de la misma reserva produzcan una única transición de estado y un único email, en vez de una condición de carrera.
- **`Clock` de negocio separado del reloj de expiración de sesión (Inc. 4):** se detectó una colisión real de bean `Clock` entre el reloj de negocio de reservas (`America/Argentina/Buenos_Aires`, para el corte de cancelación) y un `Clock` relacionado a sesiones; resuelta con un `Supplier<Clock>` para no pisar el bean existente.
- **Cancelación como transición de estado, no soft-delete (Inc. 4):** `CANCELLED` ya existía en el enum `ReservationStatus`; la reserva se conserva completa (auditoría, historial), simplemente deja de ser accionable.
- **Nuevas queries en repositorios:**
  - `ReservationRepository.findByUserIdOrderByCheckInDesc(Long userId)`

- `ReservationRepository.existsByUserIdAndLodgingIdAndStatus(...)`
- `ReservationRepository.findAllByOrderByIdDesc()` (Inc. 2)
- `RatingRepository.countByLodgingId(Long lodgingId)`

### 2.2. Frontend (React + Vite)

```
src/
├── components/
│   ├── RequireAuth.jsx              (Inc. 1 — route guard, layout <Outlet/>)
│   ├── RequireAdmin.jsx             (Inc. 3 — reescrito como layout <Outlet/>, redirect a /unauthorized)
│   ├── SortableTh/SortableTh.jsx    (Inc. 2 — encabezado de columna ordenable)
│   ├── Pagination/Pagination.jsx    (Inc. 2 — control de paginación reutilizable)
│   ├── WhatsAppButton/WhatsAppButton.jsx (Inc. 1 — botón flotante fijo bottom-right)
│   ├── RouteChunkErrorBoundary.jsx  (Inc. 4 — nuevo, recuperación manual ante fallo de carga de chunk)
│   └── RouteLoadingFallback.jsx     (Inc. 4 — nuevo, spinner accesible con retraso de 150ms)
├── hooks/
│   └── useTableData.js              (Inc. 2 — ordenamiento y paginación cliente)
├── services/
│   ├── lodgingService.js            (Inc. 3 — nuevo)
│   ├── categoryService.js           (Inc. 3 — nuevo)
│   └── favoriteService.js           (Inc. 3 — nuevo)
├── context/
│   └── AuthContext.jsx              (Inc. 4 — bootstrap desde /me, secuencia login/registro con bootstrapCsrf antes de publicar estado)
├── pages/
│   ├── Booking/BookingPage.jsx, BookingConfirmation.jsx (Inc. 1)
│   ├── MyReservations/MyReservationsPage.jsx (Inc. 1 — historial; Inc. 4 — cancelación self-service)
│   ├── Unauthorized/Unauthorized.jsx (Inc. 3 — nuevo)
│   ├── SearchResults/SearchResults.jsx (Inc. 2 — Pagination unificada; Inc. 3 — reescrito server-driven)
│   └── Admin/
│       ├── AdminCategories.jsx, AdminFeatures.jsx, AdminPolicies.jsx, AdminUsers.jsx (Inc. 2 — useTableData)
│       ├── AdminLodgings.jsx, LodgingsTable.jsx (Inc. 2 — migradas a paginación cliente)
│       └── AdminDashboard.jsx (Inc. 2 — sección "Últimas reservas")
```

- **BookingPage en dos columnas (Inc. 1):** columna izquierda con datos del alojamiento, columna derecha con formulario y cálculo dinámico del total.
- **RequireAuth como route guard (Inc. 1):** redirige a `/login` preservando la ruta destino en el state ( `from` ).
- **`useTableData` (Inc. 2):** encapsula página actual, ordenamiento ( `sortKey` , `direction` ) y filtrado; provee `paginatedData` y handlers.
- **`SortableTh` / `Pagination` (Inc. 2):** aíslan la lógica visual, reutilizados en distintas partes del sistema.
- **Migración histórica a client-side en Alojamientos (Incremento 2):** durante ese incremento, `AdminLodgings` pasó a una petición plana `GET /api/lodgings` más `useTableData` local, en línea con el resto de las entidades administrativas. La implementación actual usa consultas server-driven; ver la Sección 9.
- **Capa de servicios por dominio (Inc. 3):** `lodgingService.searchLodgings(params)` , `categoryService.getCategories()` , `favoriteService.getFavorites()/addFavorite()/removeFavorite()` — desacoplan las llamadas HTTP directas de la UI. El alcance de esta primera capa se limitó a `SearchResults.jsx` como slice experimental (no se tocaron `FavoritesPage.jsx` / `ProductCard.jsx` , que siguen llamando `api.js` directo).
- **`SearchResults.jsx` server-driven (Inc. 3):** se eliminó la paginación local ( `page` / `PAGE_SIZE=9` + slicing) y el filtrado en memoria de múltiples categorías ( `runCategorySearch` ); ahora un único `runSearch()` siempre envía las categorías seleccionadas al backend y la paginación se dirige por `currentPage` / `totalPages` de la respuesta. Los clics de paginación disparan un refetch real (vía `lastSearchRef` , que preserva los filtros aplicados), no un slice local.
- **`RequireAdmin` como layout (Inc. 3):** retorna `<Outlet/>` en vez de envolver `children` , igual que `RequireAuth` . El redirect para usuario autenticado sin rol ADMIN cambia de `/` a `/unauthorized` (página nueva, mínima, reutiliza estilos existentes de `App.css` ).
- **11 páginas de rutas cargadas con `React.lazy()` (Inc. 4):** reduce el bundle inicial descargado por el cliente.
- **`RouteChunkErrorBoundary` (Inc. 4):** ante un fallo de carga de chunk (típico tras un deploy con caché de chunks obsoleta), ofrece recarga manual completa en vez de una pantalla en blanco; se resetea automáticamente al cambiar de ruta (navegar fuera y volver reintenta sin recargar toda la página).
- **`RouteLoadingFallback` con retraso de 150ms (Inc. 4):** evita el parpadeo del spinner en navegaciones que resuelven casi instantáneamente; respeta `prefers-reduced-motion` .
- **Metadata en español acotada a lo estático (Inc. 4):** se evaluó agregar títulos dinámicos por ruta ( `react-helmet` u otra dependencia), pero se descartó por sobredimensionar la solución para el alcance real — `index.html` con `lang="es"` y `<title>` fijo cubre el requisito.
- **Cancelación desde `MyReservationsPage.jsx` (Inc. 4):** botón visible solo para reservas `CONFIRMED` con check-in futuro (misma regla que valida el backend); confirmación nativa ( `window.confirm` ), bloqueo de doble envío por fila mientras la solicitud está en curso, y reemplazo solo de la fila afectada al finalizar (sin refetch completo).

## 3. Trazabilidad de Historias de Usuario (User Stories)

| ID | HISTORIA DE USUARIO | COMPONENTE / VISTA UI | ENDPOINT BACKEND | CRITERIO DE ACEPTACIÓN / ESTADO |
|---|---|---|---|---|

| | | | | |
|---|---|---|---|---|
| **US #30** | Seleccionar fecha de check-in/check-out con fechas ocupadas bloqueadas. | `BookingPage.jsx` | `GET /api/lodgings/{id}/availability` | Calendario con rangos CONFIRMED deshabilitados cargados al montar la página. |
| **US #31** | Visualizar detalles del alojamiento al iniciar la reserva. | `BookingPage.jsx` | `GET /api/lodgings/{id}` | Nombre, ciudad, precio, imagen, descripción y features visibles en columna izquierda. |
| **US #32** | Realizar la reserva y recibir confirmación en pantalla. | `BookingPage.jsx`, `BookingConfirmation.jsx` | `POST /api/reservations` | HTTP 201 → redirige a `/booking/confirmation` con nombre, fechas, huésped y total. |
| **US #33** | Acceder al historial personal de reservas. | `MyReservationsPage.jsx` | `GET /api/reservations/my` | Lista ordenada por `checkIn DESC`. Protegida por `RequireAuth`. |
| **US #34** | Iniciar conversación de WhatsApp con mensaje pre-cargado. | `WhatsAppButton.jsx` | N/A (Frontend) | Botón flotante visible para todos. Oculto si `VITE_WHATSAPP_NUMBER` no definida. |
| **US #35** | Recibir email de confirmación al realizar una reserva. | `BookingConfirmation.jsx` | `POST /api/reservations` (dispara `EmailService`) | `SmtpEmailServiceImpl` envía email HTML. `ConsoleEmailServiceImpl` loguea por defecto. |
| **US #36** | Visualizar tablas administrativas uniformes con ordenación y paginación local. | `AdminCategories`, `AdminFeatures`, `AdminPolicies`, `AdminUsers`, `AdminLodgings` | N/A (Frontend) | Tablas con estilos homogéneos, indicadores visuales de ordenación y paginado consistente. |
| **US #37** | Acceder a estadísticas de reservas y a la sección de reservas recientes. | `AdminDashboard.jsx` | `GET /api/reservations` | Tarjeta de estadística + tabla "Últimas reservas" (4 más recientes). Acceso restringido a ADMIN. |
| **US #38** | Buscar alojamientos filtrando por múltiples categorías, con resultados paginados desde el servidor. | `SearchResults.jsx` | `GET /api/lodgings/search` | Selección de 2+ categorías filtra en base de datos (no en memoria). Paginación dirigida por `currentPage` / `totalPages` del servidor. |
| **US #39** | Recibir mensajes de error en español o inglés según el idioma del navegador. | N/A (Backend, transversal) | Cualquier endpoint que dispare `ResourceNotFoundException`, `IllegalArgumentException`, o validación de `page` / `size` en `/search` | Header `Accept-Language: es` → mensaje en español; `en` o ausente → inglés. Logs siempre en inglés. |
| **US #40** | Acceso uniforme a rutas administrativas con redirección clara cuando falta permiso. | `RequireAdmin.jsx`, `Unauthorized.jsx` | N/A (Frontend) | Usuario autenticado sin rol ADMIN es redirigido a `/unauthorized` (antes: `/`, sin explicación). |
| **US #41** | Iniciar sesión sin exponer el JWT a JavaScript, protegido contra CSRF. | `AuthContext.jsx`, `api.js` | `POST /api/auth/login`, `GET /api/auth/csrf`, `GET /api/auth/me` | `ACCESS_TOKEN` en cookie `HttpOnly`; UI no publica sesión autenticada hasta que el bootstrap de CSRF resuelve. |
| **US #42** | Cancelar una reserva propia antes de la fecha de check-in. | `MyReservationsPage.jsx` | `PATCH /api/reservations/{id}/cancel` | Botón visible solo si `CONFIRMED` + check-in futuro; confirmación previa; reserva ajena o ya iniciada rechazada. |
| **US #43** | Que un fallo de carga de una página no deje la aplicación en blanco. | `RouteChunkErrorBoundary.jsx`, `RouteLoadingFallback.jsx` | N/A (Frontend) | Fallo de chunk muestra mensaje y opción de recarga; navegación normal muestra spinner solo si tarda más de 150ms. |

## 4. Catálogo de Endpoints Nuevos / Modificados

### 4.1. Reservas

| MÉTODO | ENDPOINT | ACCESO (RBAC) | DESCRIPCIÓN |
|---|---|---|---|
| POST | `/api/reservations` | Autenticado | Crear reserva (incluye `guestPhone`) |
| GET | `/api/reservations/my` | Autenticado | Historial del usuario autenticado, ordenado por `checkIn DESC` |
| GET | `/api/reservations` | ADMIN | Todas las reservas del sistema, ordenadas por `id DESC` (Inc. 2) |
| GET | `/api/lodgings/{id}/availability` | Público | Disponibilidad. Sin params: `occupiedRanges`. Con `checkIn` / `checkOut`: además `available`. |
| PATCH | `/api/reservations/{id}/cancel` | Autenticado | **(Inc. 4, nuevo)** Cancela una reserva propia `CONFIRMED` antes del check-in (evaluado con reloj de negocio). Idempotente si ya está `CANCELLED`; 400 si el check-in ya llegó; 404 si es ajena o no existe. |

### 4.2. Alojamientos

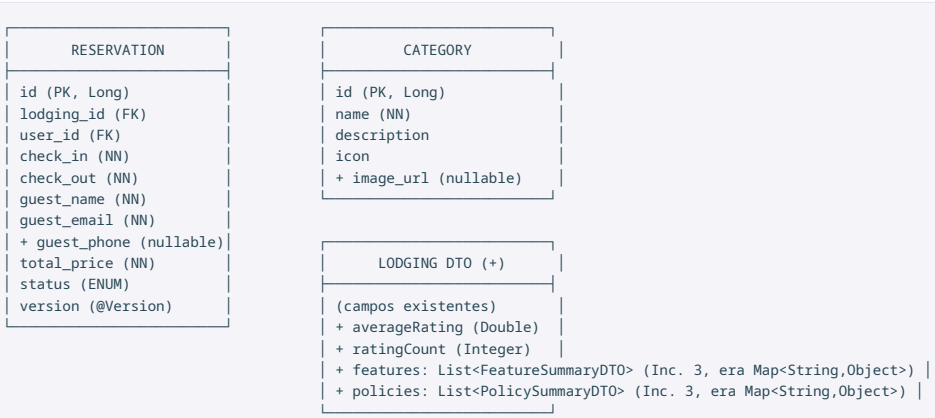
| MÉTODO | ENDPOINT | ACCESO (RBAC) | DESCRIPCIÓN |
|---|---|---|---|
| PUT | `/api/lodgings/{id}` | ADMIN | Usado desde `LodgingFormModal` en modo edición. |
| GET | `/api/lodgings/{id}` | Público | `LodgingDTO` incluye `averageRating`, `ratingCount`, y (Inc. 3) `features` / `policies` tipados. |
| GET | `/api/lodgings/search` | Público | **(Inc. 3, breaking change de contrato)** Acepta `categories` (múltiple), `page`, `size` (validados). Retorna `{lodgings, currentPage, totalItems, totalPages}` en vez del array plano anterior. |

### 4.3. Autenticación (Inc. 4)

| MÉTODO | ENDPOINT | ACCESO (RBAC) | DESCRIPCIÓN |
|---|---|---|---|
| POST | `/api/auth/register` | Público, CSRF exento | Crea el usuario y entrega `ACCESS_TOKEN` en cookie `HttpOnly`; rota el token CSRF si había uno previo. |
| POST | `/api/auth/login` | Público, CSRF exento | Autentica y entrega `ACCESS_TOKEN` en cookie `HttpOnly`; el cuerpo no expone el JWT. |
| POST | `/api/auth/logout` | Público (requiere CSRF válido) | Limpia `ACCESS_TOKEN`. 204 idempotente; 403 si el CSRF es inválido o falta. |
| GET | `/api/auth/me` | Autenticado (401 si no) | Identidad de la sesión vigente. |
| GET | `/api/auth/csrf` | Autenticado (401 si no) | Bootstrap explícito del token CSRF; 204, sin body. |

## 5. Modelo de Datos

### Modificaciones en Entidades y DTOs




- **Reservation:** `guestPhone` (String, nullable) para el formulario de reserva y contacto WhatsApp.
- **Category:** `imageUrl` (String, nullable) para `CategoryCard` y secciones de búsqueda.
- **LodgingDTO:** `averageRating` / `ratingCount` se calculan en `LodgingServiceImpl.enrichWithRatings()`, no son columnas persistidas.
- **`FeatureSummaryDTO` / `PolicySummaryDTO` (Inc. 3, nuevos):** `{id, name, icon}` y `{id, name, description, icon}` respectivamente. Reemplazan los `Map<String, Object>` genéricos, con mapeo manual ( `fromEntity` ) — se descartó MapStruct por directiva del proyecto.
- **`ResourceNotFoundException` (Inc. 3):** se agregaron campos opcionales `errorCode` / `args` (retrocompatibles) para soportar localización por clave en el futuro; el único sitio real que la lanza hoy ( `ReservationServiceImpl` ) sigue usando el constructor de mensaje plano.
- **Bundles de mensajes (Inc. 3, nuevos):** `messages.properties` / `messages_es.properties` con únicamente las claves que dispara código real: `error.page.negative`, `error.size.negative`, `error.resource.not_found`.
- **`Reservation.status = CANCELLED` (Inc. 4):** el valor ya existía en el enum desde el sprint original, pero ningún flujo de usuario lo alcanzaba; la cancelación self-service lo hace accionable por primera vez. No hay borrado físico ni columna nueva.
- **`RefreshToken` / `RefreshTokenFamily` / `SessionSecurityEvent` (Inc. 4, nuevas, migración `V2__refresh_session_families.sql`):** hashing de tokens ( `RefreshTokenHasher` ), agrupación por familia para detectar reuso/replay, y bitácora de eventos de seguridad de sesión. Entidades completas y probadas de forma aislada, pero **sin ningún endpoint que las use** — ver Sección 8.

### Nuevas Queries en Repositorios

| REPOSITORIO | MÉTODO | PROPÓSITO |
|---|---|---|

| | | |
|---|---|---|
| `ReservationRepository` | `findByUserIdOrderByCheckInDesc(Long)` | Historial de reservas del usuario |
| `ReservationRepository` | `existsByUserIdAndLodgingIdAndStatus(Long, Long, Status)` | Valida reserva CONFIRMED antes de permitir puntuación |
| `ReservationRepository` | `findAllByOrderByIdDesc()` | (Inc. 2) Listado admin de todas las reservas |
| `RatingRepository` | `countByLodgingId(Long)` | Conteo de reseñas para `ratingCount` |

## 6. Decisiones Técnicas Clave

- **Página dedicada vs modal para reservas (Inc. 1):** el criterio de la US #31 requiere que la información completa del alojamiento sea visible durante la reserva, difícil de lograr en un modal sin saturarlo.
- `occupiedRanges` **sin parámetros de fecha (Inc. 1):** permite que `BookingPage` bloquee fechas al cargar, sin esperar que el usuario seleccione un rango primero.
- `@Primary` **sobre** `@ConditionalOnMissingBean` **para EmailService (Inc. 1):** comportamiento determinista garantizado; el toggle se controla con `app.mail.smtp.enabled`.
- **WhatsApp como enlace** `wa.me` **(Inc. 1):** evita la dependencia de la API oficial de WhatsApp Business (cuenta verificada, aprobación de Meta).
- **Edición de alojamiento reutilizando** `LodgingFormModal` **(Inc. 1):** prop `lodging` opcional en vez de un componente separado, minimiza duplicación.
- **Ordenamiento y paginación en el cliente para tablas de administración (Inc. 2):** volúmenes de datos bajos/medios en un contexto exclusivamente admin — carga única simplifica el backend y da experiencia inmediata. Documentado explícitamente como reversible "si la volumetría de producción lo requiere" (ver Inc. 3 para el caso donde sí se justificó — búsqueda pública, no tablas admin).
- **Estandarización de** `SortableTh` / `Pagination` **(Inc. 2):** reutilizables entre distintas partes del sistema, incluida `SearchResults`.
- **TDD en la sección de reservas del dashboard (Inc. 2):** se implementó primero `ReservationControllerIntegrationTest` para asegurar `hasRole('ADMIN')` antes de construir servicio y UI.
- **Reutilizar el formato de respuesta paginada existente para** `/search` **(Inc. 3):** en vez de introducir un tercer formato de paginación, se adoptó el mismo `Map` ( `lodgings`, `currentPage`, `totalItems`, `totalPages` ) que ya usaba `GET /api/lodgings` — a costa de ser un cambio de contrato para `/search`, con un único consumidor conocido ( `SearchResults.jsx` ), actualizado en el mismo cambio.
- `@Validated` + `@Min(message="{clave}")` **en vez de excepción-con-mensaje-como-clave (Inc. 3):** el patrón de localización originalmente propuesto para `IllegalArgumentException` (tratar `ex.getMessage()` como clave de traducción) no es compatible con ningún punto real del código existente. Se usó Bean Validation, el mecanismo nativo de Spring para esto.
- **Alcance acotado de la i18n a 4 de 9 exception handlers (Inc. 3):** decisión explícita para no inflar el diff tocando código no relacionado con la búsqueda/paginación; documentado como deuda técnica controlada (Sección 8).
- `RequireAdmin` **como layout** `<Outlet/>` **(Inc. 3):** alinea con el patrón ya usado por `RequireAuth`, en vez de mantener dos convenciones distintas de route guard.
- **Cookie** `HttpOnly` **+ CSRF sobre JWT en** `localStorage` **(Inc. 4):** el JWT queda inaccesible desde JavaScript, mitigando exfiltración por XSS, a cambio de requerir protección CSRF explícita en las mutaciones (detalle en 1.4.1 y 2.1).
- `NullAuthenticatedSessionStrategy` **en vez del** `CsrfAuthenticationStrategy` **default (Inc. 4):** el default rota el CSRF en cada request autenticado bajo `STATELESS` (no solo login), causando cierres de sesión intermitentes; corregido con una rotación acotada y atómica en login/registro (detalle en 1.4.1).
- **Sesiones renovables entregadas como base aislada, no integrada (Inc. 4):** se priorizó una base de persistencia/rotación/replay-detection completa y probada por sobre una integración parcial al flujo HTTP real; la conexión queda para un incremento futuro (ver Sección 8).
- **Lock pesimista para cancelación de reservas (Inc. 4):** `findByIdForUpdate` evita que cancelaciones concurrentes de la misma reserva produzcan más de una transición de estado o más de un email.
- `Clock` **de negocio en vez del reloj del sistema (Inc. 4):** el corte de cancelación se evalúa contra `America/Argentina/Buenos_Aires`, no la zona horaria del servidor — corrige además un test intermitente en CI que usaba el reloj del sistema.
- **Confirmación nativa (** `window.confirm` **) para cancelar, no un modal custom (Inc. 4):** acción destructiva poco frecuente, sin justificación para un componente dedicado en este alcance.
- `RouteChunkErrorBoundary` **con recarga completa, no reintento silencioso (Inc. 4):** un fallo de chunk suele deberse a una versión de build obsoleta en caché; recargar la página entera garantiza obtener los chunks vigentes, a diferencia de un reintento in-place que podría repetir el mismo fallo.
- **Sin** `react-helmet` **para metadata (Inc. 4):** el alcance real (un `<title>` estático en español) no justificaba una dependencia adicional para títulos dinámicos por ruta.

## 7. Testing

### 7.1. Cobertura Automatizada

- **Backend — histórico:** 284 tests en el cierre original y 381/381 verificados en CI sobre el commit de integración a `main`. **Estado actual:** 422/422 pasaron en CI sobre `main` (JUnit 5 + Mockito, integración con MockMvc + Testcontainers/MariaDB 10.11). Incluye, entre otros:
  - Incremento 1: `ReservationServiceImplTest`, `ReservationControllerIntegrationTest`, `ReservationOwnershipIntegrationTest` (5 escenarios IDOR), `ReservationConcurrencyTest`.

- Incremento 2: `ReservationControllerIntegrationTest` — RBAC de `GET /api/reservations` (401/403/200) y ordenamiento `id DESC`.
- Incremento 3: `LodgingDTOTest` (mapeo a DTOs tipados), `LodgingServiceImplTest` / `LodgingControllerIntegrationTest` (paginación, filtro multi-categoría, validación de `page` / `size`), `GlobalExceptionHandlerTest` (localización de los 4 handlers en scope), `ReservationControllerIntegrationTest` (localización del caso real de `ResourceNotFoundException`).
- Incremento 4: `AuthCsrfLifecycleIntegrationTest` (7 tests — materialización del token, rotación atómica en login/registro, rechazo de token faltante/mismatcheado), `AuthControllerIntegrationTest`, `AuthCookieFactoryTest`, `JwtAuthenticationFilterIntegrationTest`, `RefreshSessionConfigurationTest` / `RefreshSessionFoundationIntegrationTest` / `RefreshSessionServiceTest` / (base de sesiones renovables, aislada), `ReservationCancellationServiceTest`, `ReservationCancellationConcurrencyTest`, 6 casos nuevos en `ReservationControllerIntegrationTest` para cancelación.

- **Frontend — histórico:** 276 tests en el cierre original y 326/326 en 46 archivos verificados en CI sobre el commit de integración a `main`. **Estado actual:** 416/416 pasaron en 53 archivos en CI (Vitest + React Testing Library). Incluye, entre otros:
  - Incremento 1: `RequireAuth.test.jsx`, `BookingPage.test.jsx`, `BookingConfirmation.test.jsx`, `MyReservationsPage.test.jsx`, `Header.test.jsx`, `WhatsAppButton.test.jsx`.
  - Incremento 2: `useTableData.test.js`, `Pagination.test.jsx`, `AdminCategories/Features/Policies/Users/Lodgings.test.jsx`, `AdminDashboard.test.jsx`, `AdminReservations.test.jsx`.
  - Incremento 3: `lodgingService.test.js`, `categoryService.test.js`, `favoriteService.test.js` (31 tests nuevos), `SearchResults.test.jsx` (reescrito — filtrado server-side, respuesta paginada), `RequireAdmin.test.jsx` (redirect a `/unauthorized`), `Unauthorized.test.jsx` (nuevo).
  - Incremento 4: `AuthContext.test.jsx` (reescrito), `AuthContextCsrfRace.test.jsx` (nuevo — secuenciación y condiciones de carrera del bootstrap CSRF), `HeaderCsrf.test.jsx`, `api.csrf.test.js`, `RouteChunkErrorBoundary.test.jsx` (3 casos), `RouteLoadingFallback.test.jsx` (4 casos, fake timers), `documentMetadata.test.jsx`, 4 casos nuevos en `MyReservationsPage.test.jsx` para cancelación.

- **Suite E2E con Playwright — histórico:** creció de 17 a 45 escenarios a lo largo de los cuatro incrementos (13 specs, incluyendo cobertura de administración de categorías/características/políticas/usuarios/alojamientos/reservas agregada progresivamente y no documentada individualmente hasta este reporte, más `verify-cookie-auth.spec.js`, nuevo en el Incremento 4). **Estado actual:** en CI run `31435735979`, desktop Chromium y Firefox registraron 44 aprobados y 1 omitido cada uno; `mobile-chromium` registró 5 aprobados en total.

### 7.2. Hallazgos Durante los Incrementos 3 y 4 (verificación empírica antes de asumir)

- El handler de `HandlerMethodValidationException` se mantiene registrado en `GlobalExceptionHandler` de forma defensiva, pero ningún endpoint real del proyecto lo dispara hoy — la validación de `page` / `size` vía `@Validated` a nivel de clase produce `ConstraintViolationException` (verificado empíricamente), no `HandlerMethodValidationException`.
- Sin un handler para esas excepciones, la validación regresaba HTTP 500 en vez de 400 (el catch-all `Exception.class` preexistente intercepta antes que la resolución nativa de Spring) — corregido con un handler dedicado.
- `spring.messages.fallback-to-system-locale=true` (default) producía resolución de idioma no determinística según el locale del sistema operativo del host — corregido explícitamente.
- La estrategia por defecto de Spring Security (`CsrfAuthenticationStrategy`) rota la cookie CSRF en cada request autenticado bajo `STATELESS`, no solo en el login — verificado leyendo el código fuente de Spring Security 6.5 y reproducido con `curl` antes de asumir la causa raíz de un flake intermitente de logout en Firefox. Una primera corrección (limpiar y regenerar el token en dos pasos) escribía dos encabezados `Set-Cookie` para el mismo nombre y rompía el siguiente logout — detectado con un test real que falló, no con inspección de código.
- Colisión de bean `Clock` entre el reloj de negocio de reservas y un `Clock` relacionado a sesiones — resuelta con un `Supplier<Clock>` dedicado para no pisar el bean existente.
- Un test de cancelación construía su fixture con `LocalDate.now()` del sistema en vez del reloj de negocio; cerca de la medianoche en Buenos Aires (UTC-3) ambos relojes podían discrepar de fecha calendario, produciendo un fallo intermitente en CI — corregido inyectando el mismo `Clock` que usa la aplicación.

## 8. Limitaciones Conocidas y Deuda Técnica Controlada

### 8.1. Deuda vigente

1. **Paginación client-side en parte del panel administrativo:** Categorías, Características, Políticas y Usuarios mantienen paginación y ordenamiento del lado del cliente. `AdminLodgings` y `AdminReservations` ya usan contratos server-driven.

2. **WhatsApp Business API:** el enlace `wa.me` no provee confirmación de envío ni manejo de errores desde la aplicación.
3. **Email SMTP desactivado por defecto:** requiere `MAIL_SMTP_ENABLED=true` y credenciales del proveedor SMTP.
4. **Precios por temporada:** el total de reserva es `días × pricePerNight`, sin tarifas variables por temporada o fin de semana.
5. **Sesiones renovables (deuda histórica del reporte original):** en el corte original, el JWT expiraba después de 8 horas sin renovación automática y la base de sesiones renovables no estaba conectada a ningún endpoint. Esta deuda se resolvió posteriormente; ver la Sección 9.
6. **Gestión de reservas en admin:** las consultas administrativas son server-driven, y desde el Incremento 4 el propio cliente puede cancelar su reserva `CONFIRMED` antes del check-in. Siguen sin existir acciones administrativas de cancelación/reprogramación, ni reprogramación de fechas para el cliente.
7. `HandlerMethodValidationException` **sin cobertura por request real:** el handler existe de forma defensiva, pero ningún endpoint actual lo dispara (ver 7.2); está cubierto solo por test unitario directo.
8. **Producción visual pendiente:** las 38 identidades están aprobadas, pero faltan generar, revisar, publicar e integrar sus cinco escenas canónicas por alojamiento, para un total de 190 imágenes.
9. **Cancelación sin lógica de reembolso:** la reserva pasa a `CANCELLED`, pero no hay integración de pagos ni reembolso — el proyecto no procesa pagos reales en ningún flujo.
10. **Metadata de documento estática:** `index.html` define `lang="es"` y un `<title>` fijo; no hay títulos ni meta-descripciones dinámicas por ruta (decisión explícita, no una limitación técnica — ver 1.4.4).
11. **Cobertura E2E creció sin registro incremental:** la suite pasó de 5 a 13 specs (17 a 45 escenarios) a lo largo de los incrementos 2 a 4 sin que cada adición quedara documentada en su momento en este reporte; este documento la consolida por primera vez (Sección 7.1).

### 8.2. Deuda registrada originalmente y resuelta después

| TEMA ORIGINAL | ESTADO VIGENTE | EVIDENCIA PRINCIPAL |
|---|---|---|
| `AdminLodgings` con paginación client-side | Resuelto; usa paginación y búsqueda desde el servidor | `LodgingController`, `LodgingServiceImpl`, `AdminLodgings.jsx` |
| Dashboard/listado de reservas sin contrato server-driven | Resuelto para consulta, filtros, ordenamiento y paginación; las mutaciones siguen pendientes | `ReservationController`, `ReservationSpecifications`, `AdminReservations.jsx` |
| i18n limitada a parte de `GlobalExceptionHandler` | Resuelto para las respuestas restantes | `GlobalExceptionHandler`, `messages.properties`, `messages_es.properties` |
| CI sin disparadores para `sprint-4` | Resuelto en configuración; no se infiere un resultado remoto | `.github/workflows/ci.yml` |
| Globals de tests ausentes en ESLint y deuda de lint | Resuelto | `frontend/eslint.config.js` y correcciones posteriores de lint |
| Esquema administrado por Hibernate y datos demo acoplados | Resuelto; Flyway administra el esquema y el seed de desarrollo requiere opt-in | `db/migration/V1__baseline_schema.sql`, `db/dev/V1_9000__dev_demo_data.sql`, `DevSeedFlywayGuard` |
| Gestión de reservas en admin sin acciones de cancelación de usuarios | Resuelto para cancelación self-service del cliente; cancelación/reprogramación admin y reprogramación de cliente siguen pendientes (ver 8.1.6) | `ReservationController`, `ReservationServiceImpl.cancelReservation()`, `MyReservationsPage.jsx` |
| JWT en `localStorage`, sin protección CSRF | Resuelto; cookie `HttpOnly` + CSRF double-submit | `AuthCookieFactory`, `SpaCsrfTokenRequestHandler`, `SecurityConfig` |

## 9. Addendum de estado actual

Este addendum reconcilia el reporte histórico con `main` en el merge commit `cd2bdee76b4a6031f1ebf0cdf3539d4e30245e89`, posterior a los PR #73 y #75. La ejecución de CI `31435735979` pasó los cinco jobs publicados: backend, frontend, desktop Chromium E2E, desktop Firefox E2E y mobile Chromium E2E.

### 9.1. Sesiones renovables

La integración de sesiones renovables se completó después del corte original. Actualmente `app.session.refresh.enabled=true` es el valor predeterminado: login y registro emiten cookies `ACCESS_TOKEN` y `REFRESH_TOKEN` `HttpOnly`; `POST /api/auth/refresh` rota el refresh token y emite un nuevo access token; logout revoca únicamente la familia de refresh del dispositivo que realiza la solicitud, mientras que el cambio de contraseña revoca **todas las sesiones de refresh del usuario**. La infraestructura ya no está aislada ni deshabilitada por defecto.

### 9.2. Tablas administrativas

`AdminLodgings` y `AdminReservations` usan consultas server-driven para búsqueda o filtrado, ordenamiento y paginación. Categorías, Características, Políticas y Usuarios conservan el modelo client-side. Las cifras y escenarios del reporte original siguen siendo históricos y no deben interpretarse como una descripción de la implementación actual.

## Anotación de estado actual

Este informe preserva las decisiones y la estructura del Sprint 4 en su contexto histórico. La búsqueda vigente se renderiza en `Home`, con la URL como fuente de verdad; las referencias históricas a la vista de resultados no describen el flujo actual. Home descarta respuestas obsoletas cuando cambia o se elimina la

consulta.